Ecualizador digital de audio de 3 bandas con comunicación RTOS-GPOS

Digital 3-band audio equalizer with RTOS-GPOS communication

[Lucio Fuster][1] · [Cristian Cherro][2]

[1] UTN, FRA, Avellaneda, Buenos Aires, Argentina · ORCID: N/A
[2] UTN, FRA, Avellaneda, Buenos Aires, Argentina · ORCID: N/A

Resumen

Este trabajo presenta el diseño e implementación de un sistema integrado que combina un ecualizador digital basado en un sistema operativo en tiempo real (FreeRTOS) con un sistema operativo de propósito general (Linux). El sistema procesa señales de audio mediante un microcontrolador ESP32, dividiendo la señal en tres bandas con ajuste de ganancia. El GPOS basado en Linux permite el control remoto de los parámetros del ecualizador a través de un driver de carácter y aplicaciones de usuario. Se detallan los aspectos de hardware, software, PCB, protocolo de comandos y resultados experimentales.

Palabras clave: FreeRTOS; GPOS; Linux; DSP; Ecualizador digital; Driver de carácter; UTN FRA

Abstract

This paper presents the design and implementation of an integrated system that combines a digital equalizer based on a real-time operating system (FreeRTOS) with a general-purpose operating system (Linux). The system processes audio signals using an ESP32 microcontroller running FreeRTOS, dividing the signal into three bands with gain adjustment. The Linux-based GPOS allows remote control of the equalizer parameters through a character device driver and user applications. Hardware, software, PCB, command protocol, and experimental results are detailed.

Keywords: FreeRTOS; GPOS; Linux; DSP; Digital equalizer; Character device driver; UTN FRA

Índice

1. Introducción **3**

2. Objetivos **3**

3. Alcance **3**

4. Diagrama en bloques del sistema integrado **3**

5. Hardware **4**

5.1. ADC I2S PCM1808 4

5.2. DAC I2S PCM5102 5

5.3. LCD 20x04 6

5.4. Encoder rotativo KY-040 6

5.5. UART PC / PL2303 6

5.6. ESP32 7

5.7. Raspberry Pi 4 7

6. Software **8**

6.1. Descripción de algoritmo 8

6.2. Tareas 9

Descripción de tareas 10

6.3. Arquitectura GPOS (Linux) 11

Kernel space: driver UART basado en serdev 12

User space: aplicación de terminal y aplicación gráfica 14

7. Diseño de PCB **15**

7.1. Esquemático 15

7.2. Layout 16

7.3. Test points 16

7.4. PCB físico 17

8. Resultados y mediciones **17**

9. Anexo imagenes **18**

10. Conclusiones **19**

1. Introducción

El procesamiento digital de señales aplicado al audio es una herramienta esencial en aplicaciones modernas de entretenimiento. Este prototipo integra dos elementos importantes de la informática y electrónica digital: una primera parte que consiste en un ecualizador digital implementado sobre un microcontrolador con un sistema operativo en tiempo real (RTOS), y una segunda parte que desarrolla la comunicación entre este sistema embebido y un sistema operativo de propósito general (GPOS) como Linux. Esto incluye la implementación de un driver en espacio de kernel y una aplicación en espacio de usuario que permita controlar remotamente el ecualizador. El objetivo es demostrar la aplicación práctica de conceptos de sistemas operativos [1, 2] así como de procesamiento embebido de señales.

2. Objetivos

Diseñar un sistema de procesamiento digital de señales que permita filtrar una señal de audio en tiempo real dividiéndola en 3 bandas de frecuencia donde a cada una de esas bandas se le debe poder ajustar la ganancia en un rango de ± 10 dB y donde se debe poder ver su configuración actual. Además debe contar con: persistencia en la configuración; elementos de HMI; gestión remota de comandos; y un PCB prototipo.

3. Alcance

El alcance del sistema integrado se define en términos de los márgenes de error admisibles con respecto al comportamiento ideal esperado. A continuación se detallan las tolerancias establecidas para las principales variables:

- **Ganancia por banda:** El ajuste se realiza en pasos de 1 dB. Se considera aceptable una desviación máxima de $\pm 0{,}4$ dB respecto del valor programado.
- **Latencia de procesamiento:** El tiempo desde la adquisición de una muestra hasta la salida analógica procesada debe ser inferior a 10 ms, lo que garantiza la operación en tiempo real para aplicaciones de audio.
- **Actualización de la interfaz gráfica:** La aplicación de usuario en el GPOS debe reflejar los cambios de ganancia en la pantalla de 1 s desde que se modifica el parámetro.

Estos límites han sido definidos para asegurar que el sistema cumpla con los requisitos funcionales de la consigna y que sus desviaciones sean suficientemente pequeñas para no afectar la percepción auditiva ni la usabilidad.

4. Diagrama en bloques del sistema integrado

El sistema consta de dos subsistemas principales: el ecualizador (RTOS) y el módulo de control desde GPOS. La Figura 1 muestra el diagrama en bloques completo donde la PC/Raspberry Pi representa el GPOS.

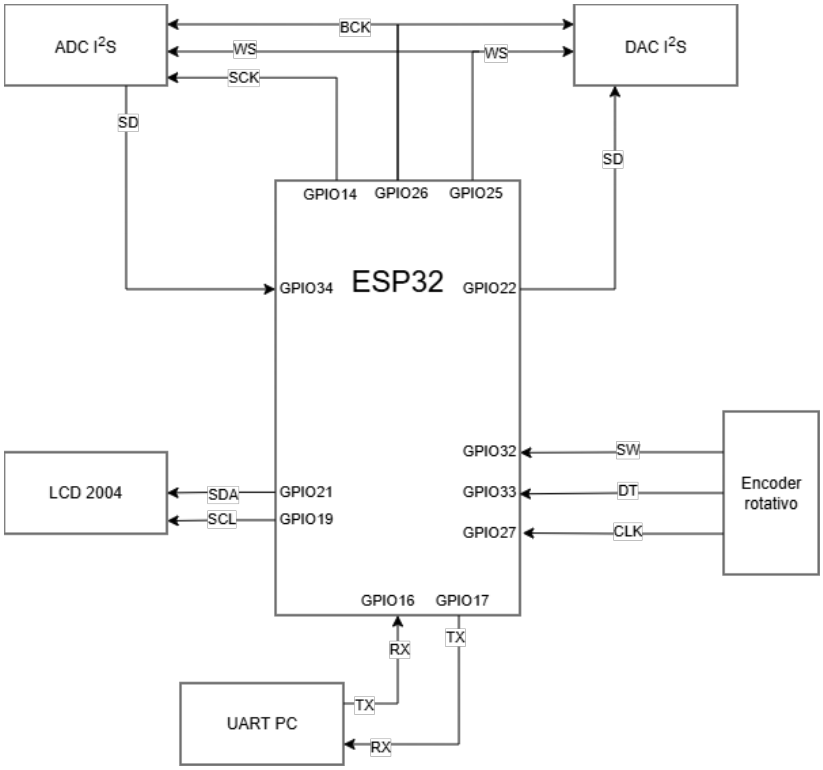


Figura 1: Diagrama en bloques del sistema integrado.

5. Hardware

5.1. ADC I2S PCM1808

El bloque funcional del ADC I2S estará construido en el PCB y está basado en el circuito integrado PCM1808 [3], es un ADC con protocolo de comunicación I2S con capacidad de funcionar en formato de 24 bits y frecuencias de muestreo que van de 8kHz a 96kHz y posibilidad de funcionar tanto como maestro o esclavo. En nuestro sistema funcionará como esclavo de la comunicación I2S, establecemos frecuencia de sampleo a 44.1kHz para muestrear la señal de entrada. Este se va a comunicar al microcontrolador mediante el hardware incluido I2S guardando los datos por DMA.



Figura 2: Integrado PCM1808

6.5 Electrical Characteristics

All specifications at T_A = 25°C, V_{CC} = 5 V, V_{DD} = 3.3 V, master mode, f_S = 48 kHz, system clock = 512 f_S, 24-bit data, unless otherwise noted

	PARAMETER	TEST CONDITIONS	MIN	TYP	MAX	UNIT
	Resolution			24		Bits
DATA FORMAT						
	Audio data interface format			I^2S, left-justified		
	Audio data bit length			24		Bits
	Audio data format			MSB-first, 2s complement		
f_S	Sampling frequency		8	48	96	kHz
	System clock frequency	256 f_S	2.048	12.288	24.576	MHz
		384 f_S	3.072	18.432	36.864	
		512 f_S	4.096	24.576	49.152	

Figura 3: Tabla de parametros PCM1808

5.2. DAC I2S PCM5102

Para el DAC I2S se usará un módulo basado en el integrado PCM5102 [4], este integrado soporta resolución de palabra de 16, 24 y 32 bit, a fin de mantener la consistencia entre ADC y DAC usaremos el formato de 24 bit. También puede soportar frecuencias de muestreo que van de 8kHz a 384kHz, la cual nuevamente se configurará a 44.1kHz por consistencia entre dispositivos. Este al igual que el ADC funcionará como esclavo de la comunicación I2S y usará los datos procesados por DSP en el microcontrolador para generar a su salida una señal analógica. Según los parámetros que da su hoja de datos tiene como máximo una salida de 2.1 Vrms con una potencia máxima de salida de 6dBm (en condición de carga mínima de 1kΩ).

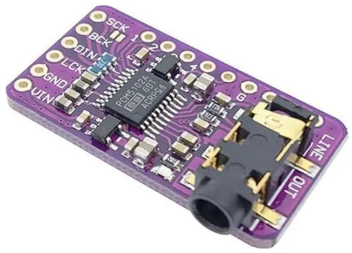


Figura 4: Módulo basado en PCM5102

	PARAMETER	TEST CONDITIONS	MIN	TYP	MAX	UNIT
	Resolution		16	24	32	Bits
Data Format (PCM Mode)						
	Audio data interface format		I^2S, left justified			
	Audio data bit length		16, 24, 32-bit acceptable			
	Audio data format		MSB First, 2's Complement			
f_S (1)	Sampling frequency		8		384	kHz
	System clock frequency		64, 128, 192, 256, 384, 512, 768, 1024, 1152, 1536, 2048, or 3072 f_{SCK}, up to 50Mhz			

Figura 5: Tabla de parámetros PCM5102

5.3. LCD 20x04

Como elemento visual de la HMI en el prototipo se usará un display LCD de 4 filas y 20 caracteres el cual indicará la configuración actual y también contará con un cursor que indica el parámetro que se quiere modificar actualmente. Este display cuenta con un módulo adaptador de comunicación por i2C basado en el integrado PCF8574T [6].



Figura 6: Frente y dorso del display LCD

5.4. Encoder rotativo KY-040

Como elemento interactivo se usa un encoder rotativo KY-040 [7].], este al ser presionado cambia la posición del cursor que indica qué parámetro se quiere modificar mientras que girarlo hará que la ganancia de la banda seleccionada por el cursor suba o baje dependiendo de si se giró en sentido horario o antihorario respectivamente.

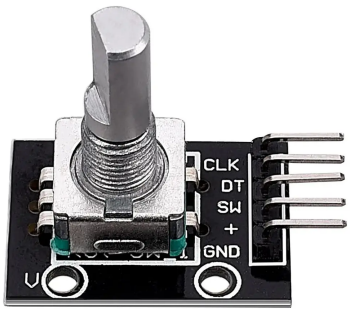


Figura 7: Encoder rotativo KY-040

5.5. UART PC / PL2303

Para configuración remota se comunica una PC (o Raspberry Pi) por UART al microcontrolador. Inicialmente para la EGA se usa un conversor USB a UART basado en PL2303 [8], para la EGB se usará una conexion cableada a los pines de la raspberry pi4.

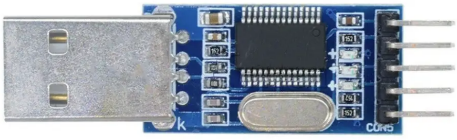


Figura 8: Adaptador basado en PL2303

5.6. ESP32

Como centro de todo el diagrama estará el microcontrolador ESP32. Este se encargará de gestionar todos los periféricos del prototipo y del procesamiento de señales, todo por medio de un programa que hace uso de FreeRTOS

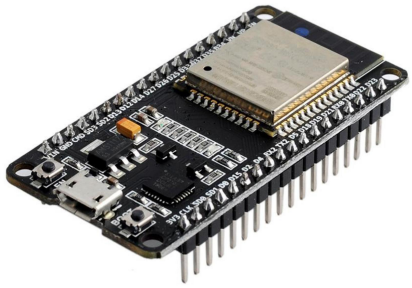


Figura 9: Placa de desarrollo ESP32

5.7. Raspberry Pi 4

Para el GPOS se utiliza una Raspberry Pi 4 como la de la figura, esta se conectará por UART al ESP32.



Figura 10: Raspberry Pi 4

6. Software

6.1. Descripción de algoritmo

Desde el punto de vista del software el microcontrolador correrá un sistema operativo en tiempo real, más específicamente FreeRTOS provisto por ESPRESIF, el programa estará compuesto por un total de 6 tareas con 4 niveles de prioridad distribuidas entre los 2 núcleos del microcontrolador.

Primero determinaremos el funcionamiento del procesamiento y luego hablaremos propiamente de las tareas. Para hacer el procesamiento de señal el sistema consiste de un triple buffer rotativo, en simultáneo suceden 3 cosas: un buffer acumula las muestras entregadas por el ADC mediante el DMA del periférico I2S; un buffer es leído y procesado por algoritmos DSP con el núcleo del ESP32 apoyándonos en la biblioteca ESP-DSP de Espressif, que proporciona funciones optimizadas para el núcleo Xtensa del ESP32, el contenido del buffer será reemplazado con la información procesada; y el último buffer envía los datos procesados al DAC mediante el DMA del periférico. Finalmente cada vez que el ADC llena un buffer y a su vez el DAC vacía el suyo (por tener frecuencias y tamaño de palabra coherentes), los buffer rotan, pasando el lleno del ADC a ser procesado, el procesado del DSP a salir por el DAC y el vaciado del DAC a ser llenado por el ADC.

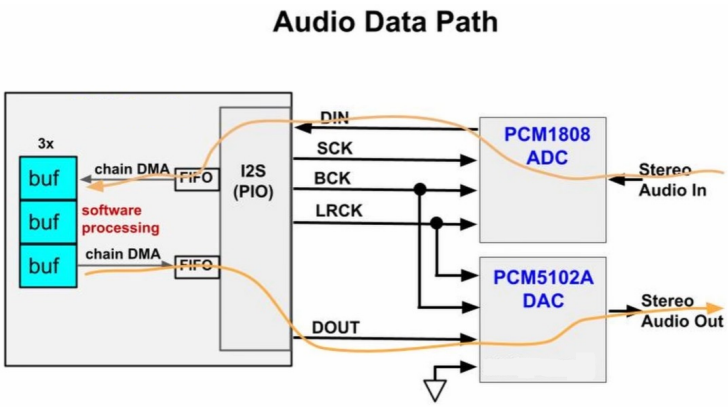


Figura 11: Datapath audio

6.2. Tareas

Como se dijo previamente el software del ecualizador implementa FreeRTOS con 6 tareas distribuidas en 2 núcleos según el siguiente diagrama de tareas:

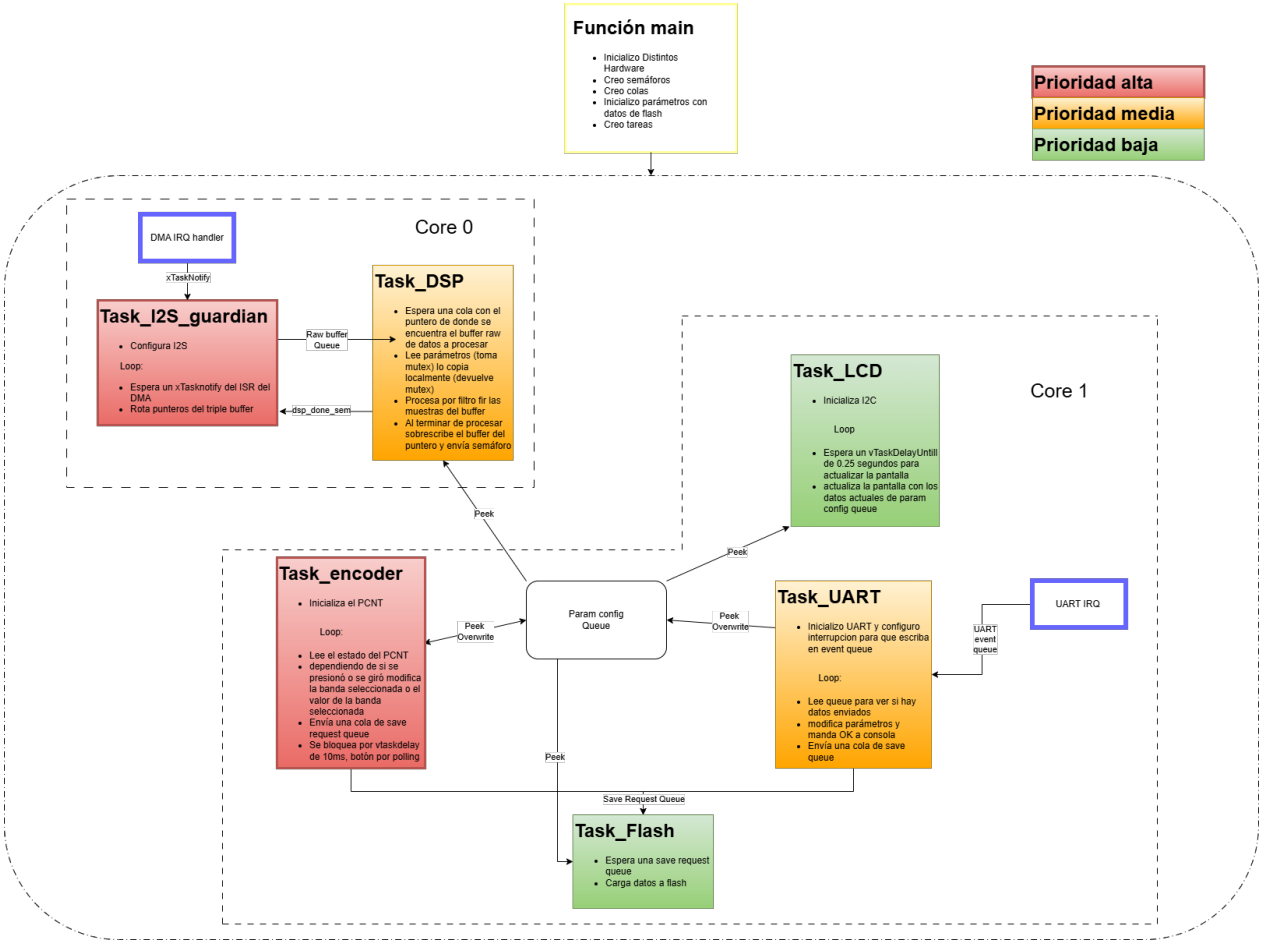


Figura 12: Diagrama de tareas del RTOS actualizado (incluye Task_UART para GPOS)

Descripción de tareas

- **Task_Guardian_I2S (prioridad alta, core 0):** es la responsable exclusiva de las operaciones de lectura escritura sobre el periférico I2S y de la rotación del triple buffer. Se despierta mediante una notificación desde la ISR de recepción del DMA. Tras leer el buffer del ADC y escribir el buffer procesado al DAC, espera a que la tarea DSP termine de procesar el buffer anterior mediante el semáforo dsp_done_sem, y luego rota los punteros. Como sincronización para despertar usa un task notification del DMA, usado por su alta velocidad frente a un semáforo [11].

- **Task_DSP (prioridad media, core 0):** su finalidad será procesar los datos provistos por el buffer del ADC por medio de filtros FIR propios de la librería ESP-DSP de Espressif, los cuales cuentan con operaciones optimizadas para el núcleo Xtensa del ESP32 [12, 13, 14]. Cada filtro tendrá sus coeficientes calculados por medio de una función dentro del programa que usa la fórmula y método provisto en la nota de aplicación de Analog Devices para la implementación digital de un filtro Windowed-Sinc [15]. Esta tarea, a la salida de los filtros, le aplicará la ganancia correspondiente según la configuración, la cual leerá con un `xQueuePeek` de la cola de acceso común `param_config_queue` para finalmente ser recombinados en una sola. La sincronización de esta tarea será por un `xQueueReceive`, la cola proviene de la tarea guardiana del I2S y cuyo contenido es un puntero al buffer donde están los datos que envió el ADC, y le entregará un semáforo binario a la tarea guardiana para indicarle que finalizó de procesar los datos.

- **Task_LCD (prioridad baja, core 1):** Actualiza la pantalla LCD 20 x 4 cada 250ms. Lee la configuración actual de param_config_queue con xQueuePeek y muestra los valores de ganancia de las tres bandas, junto con un cursor (<) que indica cuál está seleccionada. La comunicación I2C se realiza con la API estándar de ESP-IDF. Esta tarea se sincroniza con un vTaskDelayUntill() que le da ese tiempo de 250ms de refresco.

- **Task_Encoder (prioridad alta, core 1):** Utiliza el módulo PCNT (Pulse Counter) provisto por espresif para el ESP32 para decodificar los pulsos de un encoder rotativo de tipo Gray. El contador se configura para incrementar o decrementar según la fase de las señales CLK y DT, eliminando la necesidad de ISRs por cada flanco. Cada vez que se acumulan una determinada cantidad de pulsos, se modifica la ganancia de la banda seleccionada (en pasos de 1dB) y se escribe el nuevo estado en param_config_queue mediante xQueueOverwrite. El pulsador (SW) se lee por polling con un antirrebote de 200ms y cambia la banda activa. Tiene un bloqueo cada 10ms con un vTaskDelay.

- **Task_UART (prioridad media, core 1):** Instala el driver UART y procesa comandos enviados desde una terminal. Los comandos tienen el formato: set <banda><valor>para el seteo de ganancia de una banda determinada, exec <comando>para ejecutar algunas macros como perfiles predeterminados o seteos multibanda y get <banda>que retorna valor actual en dB de la banda consultada. En base a lo recibido realizará las modificaciones indicadas en

"param config queue" mediante xQueueOverwrite. La sincronización de esta tarea será por medio de un xQueueReceive de una cola llamada "UART_event_queue" que contendrá la información enviada desde la terminal y en base a ello actuará. Esta tarea también entregará el semáforo llamado "change_semaphore" que indica que se hizo un cambio en la cola "param config queue".

- **Task_Flash (prioridad baja, core 1):** su finalidad será tomar los parámetros actuales de "param config queue" y escribirlos en la memoria FLASH del microcontrolador para darle permanencia a la configuración entre apagados. La sincronización de esta tarea será por medio de un xSemaphoreTake de un semáforo llamado "change_semaphore", este lo entregan tareas que hayan modificar los parámetros de "param config queue".

6.3. Arquitectura GPOS (Linux)

El sistema GPOS corre sobre una Raspberry Pi 4 con kernel 6.12.25 o superior. Se divide en dos capas: **kernel space**, donde vive un módulo que maneja directamente la UART y expone un character device, y **user space**, donde corren las aplicaciones de control. La cadena completa de software es:

`ESP32 (RTOS) ↔ UART ↔ serdev ↔ esp32_link ↔ /dev/esp32link ↔ eq_ctl ↔ SSH ↔ App C#`

La Figura 13 resume la interacción entre las capas.

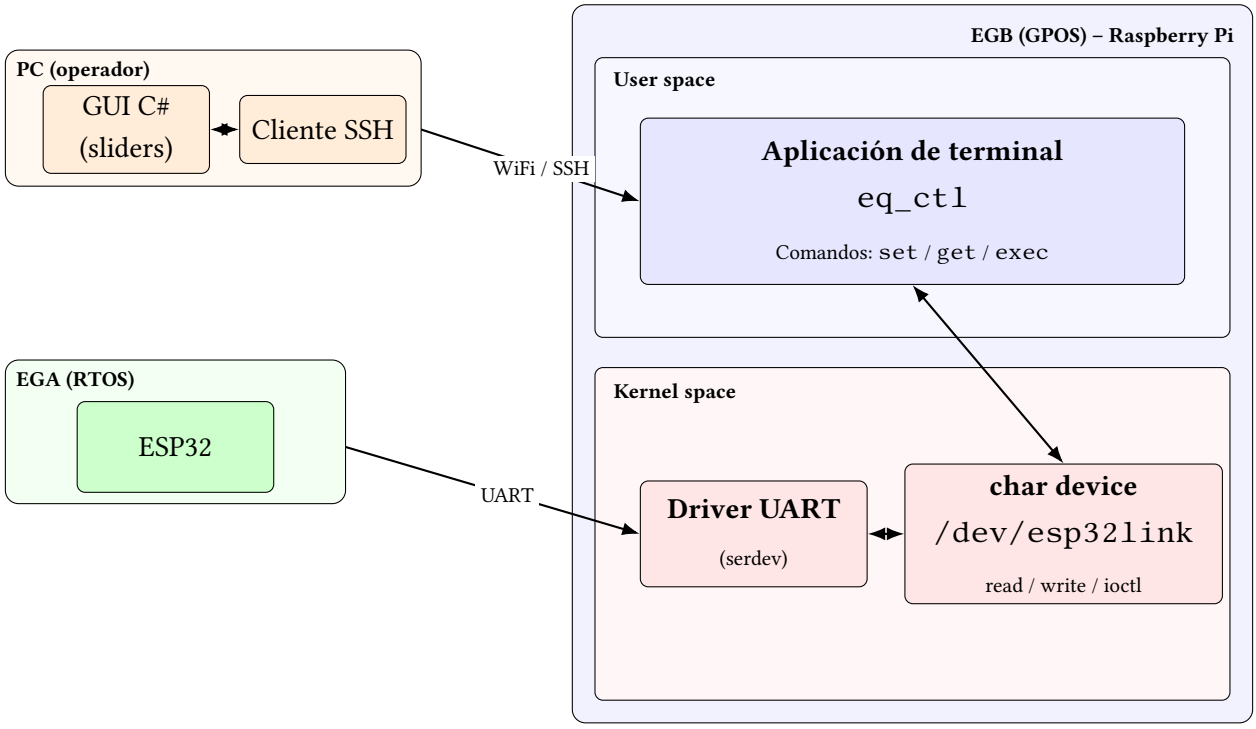


Figura 13: Diagrama de software integrado EGA+EGB.

Kernel space: driver UART basado en serdev

El módulo `esp32_link.ko` se registra como driver *serdev* asociado a la UART3 de la Raspberry Pi. Se eligió `serdev` en lugar de acceder directamente al periférico UART para:

- Integrarse con el subsistema TTY del kernel mediante el modelo estándar de dispositivos.
- Evitar colisiones con el driver nativo de la UART (`8250`).
- Delegar el manejo de pines y baudrate al Device Tree Overlay.

Estructura del driver. El módulo se declara como `serdev_device_driver` con una tabla `of_device_id` cuyo campo `compatible` es `"td3,esp32-link"`. En `probe()` se ejecutan los siguientes pasos:

1. Registro de los callbacks de serdev mediante `serdev_device_set_client_ops()`.
2. Apertura del puerto con `serdev_device_open()` y configuración de baudrate (leído del DT o 115200 por defecto).
3. Reserva del `kfifo` de recepción y de la `completion` para respuestas de `get`.
4. Reserva dinámica de major/minor, registro del `cdev` y creación de la clase y del dispositivo (`/dev/esp32link`).

El estado global del driver incluye:

- Un `kfifo` (`rx_fifo`) y una `waitqueue` (`rx_wait`) para el flujo asíncrono de `read()`.
- Una `completion` (`get_done`) y un `mutex` (`get_lock`) para el flujo síncrono de `ioctl(ESP32_GET)`.

Interfaz ioctl. La interfaz con user space se define en `esp32_link.h`. Se eligió el magic number `'y'` para evitar conflictos con otros drivers del entorno de la cátedra:

Listing 1: Interfaz ioctl (extracto de esp32_link.h)

```
#define ESP32_VAR_NAME_MAX 16

struct esp32_var {
    char nombre[ESP32_VAR_NAME_MAX];
    int  valor;
};

#define ESP32_MAGIC 'y'
#define ESP32_SET _IOW (ESP32_MAGIC, 1, struct esp32_var)
#define ESP32_GET _IOWR(ESP32_MAGIC, 2, struct esp32_var)
```

Recepción de datos. La función `esp32_receive_buf()` es llamada por el kernel cada vez que llegan bytes por la UART. Acumula los bytes en un `linebuf` hasta encontrar el carácter `\n`. Cuando completa una línea:

1. La publica en el `kfifo` y despierta a cualquier `read()` que esté esperando.
2. Si hay un ESP32_GET pendiente (flag `esperando_get`), copia la línea a `get_response` y llama a `complete(&get_done)`.

Envío de datos. La función `esp32_write()` copia los datos de usuario a un buffer local, asegura el terminador `\n` y los envía con `serdev_device_write()` con un timeout de 1 segundo.

Device Tree Overlay. El nodo `esp32-link` se declara en `esp32-link-pi4-overlay.dts` y se cuelga de `uart3`. Los pines GPIO4 y GPIO5 se configuran en ALT4 (TXD3/RXD3) mediante un grupo de pinctrl dedicado, evitando depender del overlay oficial `uart3`:

Listing 2: Extracto del Device Tree Overlay

```
fragment@0 {
    target = <&gpio>;
    __overlay__ {
        esp32_link_pins: esp32_link_pins {
            brcm,pins = <4 5>;
            brcm,function = <3>; /* ALT4: GPIO4=TXD3, GPIO5=RXD3 */
            brcm,pull = <0 2>;   /* TX sin pull, RX con pull-up */
        };
    };
};

fragment@1 {
    target = <&uart3>;
    __overlay__ {
        pinctrl-names = "default";
        pinctrl-0 = <&esp32_link_pins>;
        status = "okay";

        esp32_link: esp32-link {
            compatible = "td3,esp32-link";
            current-speed = <115200>;
            status = "okay";
        };
    };
};
```

Compilación. El Makefile del módulo genera dos artefactos: el módulo del kernel `esp32_link.ko` y el ejecutable de usuario `eq_ctl`:

Listing 3: Makefile del módulo y del cliente de usuario

```
KDIR := /lib/modules/$(shell uname -r)/build
MDIR := $(PWD)/build

obj-m += esp32_link.o

all: module ctl

module:
    @mkdir -p build; \
    cp -f esp32_link.c esp32_link.h Makefile $(MDIR); \
    cd build; make -C $(KDIR) M=$(MDIR) modules

ctl:
    gcc -Wall -pthread -o eq_ctl eq_ctl.c

clean:
    @cd build; make -C $(KDIR) M=$(MDIR) clean; cd ..; \
    rm -rf build eq_ctl
```

User space: aplicación de terminal y aplicación gráfica

Se desarrollaron dos aplicaciones de usuario que interactúan con `/dev/esp32link`:

Aplicación de terminal (`eq_ctl`). Programa en C que corre en la Raspberry Pi, abre el dispositivo y ofrece un prompt interactivo con tres comandos:

- `set BANDA NIVEL`: usa `ioctl(ESP32_SET)`. El driver lo traduce a `set BANDA NIVEL\n` por la UART.
- `get BANDA`: usa `ioctl(ESP32_GET)`. Bloquea hasta que el ESP32 responde con `BANDA=valor`.
- `exec PRESET`: usa `write()`. Se manda el texto crudo y el firmware lo interpreta como una macro multibanda.

Un hilo secundario (`hilo_monitor`) bloquea en `read()` e imprime por consola todo lo que el ESP32 envía de forma espontánea. Un mutex (`consola`) serializa las impresiones del hilo principal y del hilo receptor para evitar mezclas en `stdout`.

Aplicación gráfica (cliente en C# por SSH). La segunda aplicación se desarrolló en C# y corre en la PC del operador. Se conecta por SSH a la Raspberry Pi usando la librería `SSH.NET` y ejecuta

remotamente el programa `eq_ctl` en modo no interactivo, enviando comandos por `stdin`. La interfaz presenta:

- Tres sliders de ganancia, uno por banda (low/mid/high), con rango de ± 12 dB y paso de 1 dB.
- Botones de presets (`rock`, `jazz`, `flat`) que ejecutan `exec <preset>`.
- Un panel de estado que refresca periódicamente los valores actuales mediante `get <banda>`.
- Indicadores de conexión SSH y del estado del driver.

Esta arquitectura en dos capas de usuario permite operar el sistema tanto de forma local (terminal) como remota (interfaz gráfica).

7. Diseño de PCB

7.1. Esquemático

El esquemático completo se muestra en la Figura 14. Incluye el ESP32, ADC, DAC, LCD, encoder, UART y alimentación.

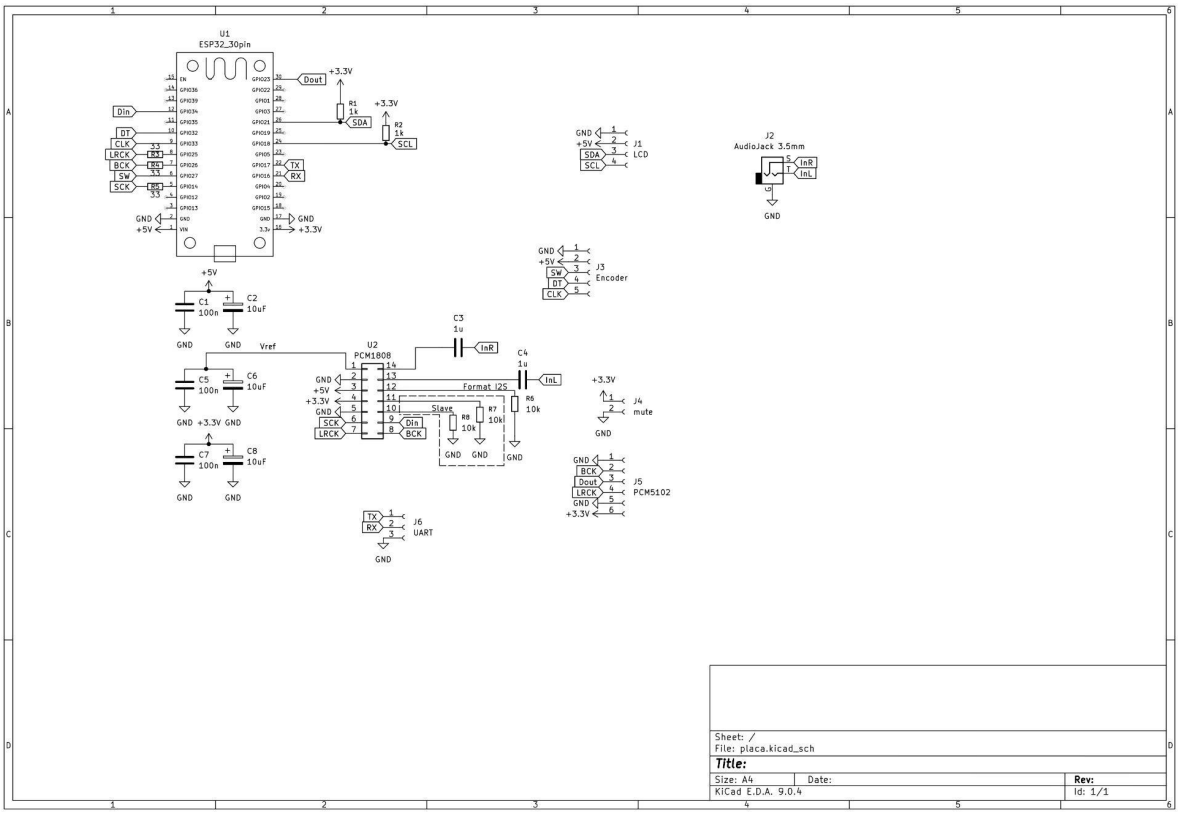


Figura 14: Esquemático del ecualizador

7.2. Layout

El layout del PCB se muestra en la Figura 16.

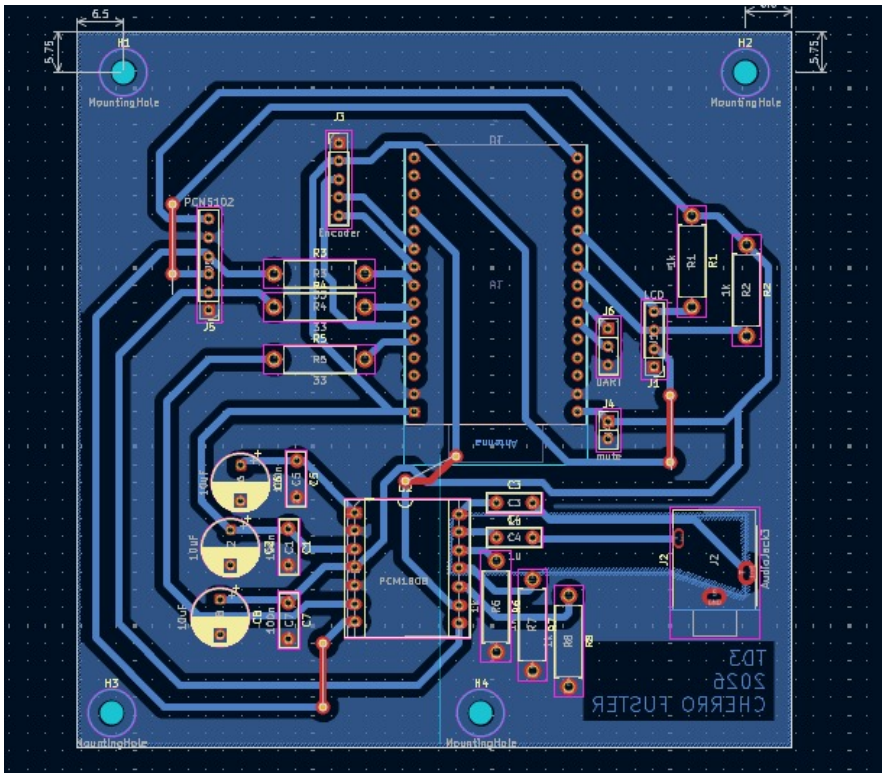


Figura 15: Layout del PCB

7.3. Test points

Test point	Señal	Descripción
TP1	Canal izquierdo IN	Entrada analógica del canal izquierdo
TP2	Canal izquierdo OUT	Salida analógica del canal izquierdo
TP3	Canal derecho IN	Entrada analógica del canal derecho
TP4	Canal derecho OUT	Salida analógica del canal derecho
TP5	GND	Masa de referencia

Cuadro 1: Lista de test points del PCB.

7.4. PCB físico

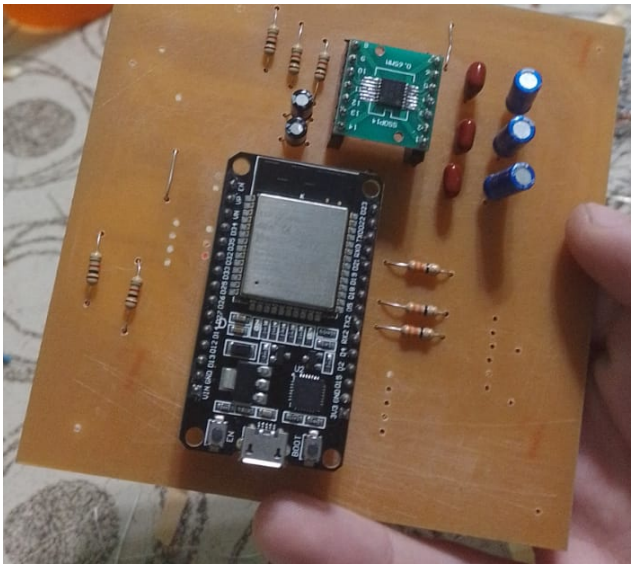


Figura 16: Producto final del PCB

8. Resultados y mediciones

El ecualizador digital demostró un procesamiento en tiempo real efectivo, con una frecuencia de muestreo de 44.1 kHz y un rango de ganancia de ± 12 dB. Las mediciones con osciloscopio confirman la respuesta de los filtros FIR. Se configuraron ganancias en +12 dB, -12 dB y 0 dB para bajos, medios y agudos respectivamente.

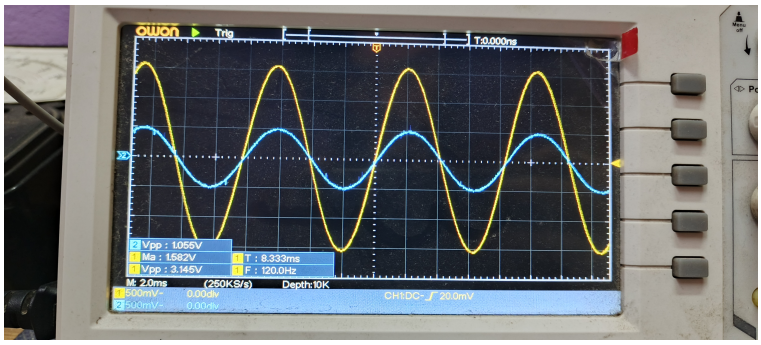


Figura 17: Captura de osciloscopio para senoidal de 120 Hz con ganancia de 10 dB

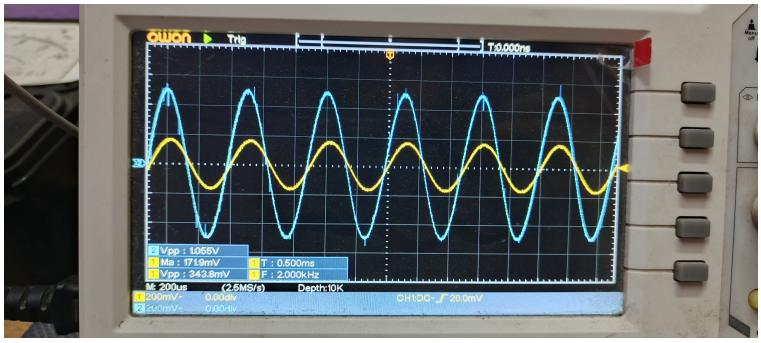


Figura 18: Captura de osciloscopio para senoidal de 2 kHz con atenuación de 10 dB

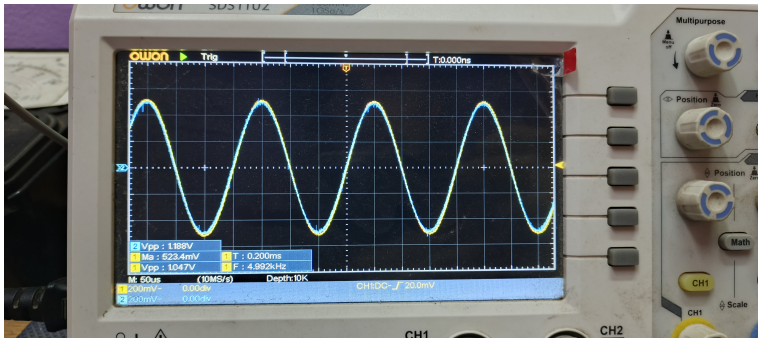


Figura 19: Captura de osciloscopio para senoidal de 5 kHz sin modificar

9. Anexo imagenes

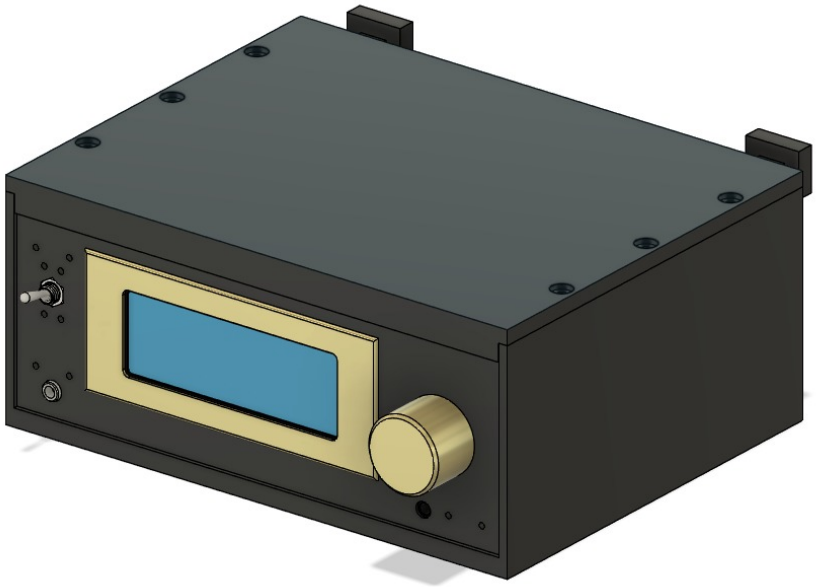


Figura 20: Frente modelo 3D Carcaza



Figura 21: Reverso modelo 3D Carcaza



Figura 22: Ensamble final

10. Conclusiones

Se ha desarrollado un sistema integrado que funciona en tiempo real y es controlable remotamente desde un sistema Linux. La interfaz gráfica proporciona una experiencia de usuario intuitiva y se lograron los objetivos propuestos por el alcance de manera satisfactoria. Dentro de las posibles mejoras al sistema estarían la inclusión de más bandas, mejorar la selectividad utilizando la version S3 del micro para aumentar la cantidad de taps del filtro y buscar un mejor aprovechamiento del rango dinámico del ADC y DAC para conseguir mejor rango que los ± 12 dB.

Referencias

[1] Intel Corporation, *80386 Programmer's Reference Manual*, Santa Clara, CA, 1990.

[2] J. Corbet, A. Rubini y G. Kroah-Hartman, *Linux Device Drivers*, 3ra ed. Sebastopol, CA: O'Reilly Media, 2005.

[3] Texas Instruments, *PCM1808 Datasheet*, 2015.

[4] Texas Instruments, *PCM5102 Datasheet*, 2014.

[5] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*, California Technical Publishing, 1997.

[6] Texas Instruments, *PCF8574T Datasheet*, 2015.

[7] KY-040 Rotary Encoder Module Datasheet.

[8] Prolific, *PL2303 Datasheet.*

[9] Raspberry Pi Ltd., *Raspberry Pi 4 Model B Hardware Specifications and GPIO Pinout*, 2023.

[10] Cátedra de Técnicas Digitales III, *Evaluación Globalizadora B: Sistemas Operativos de Propósito General*, UTN FRA, 2026.

[11] freertos Documentation, *FreeRTOS direct to task notifications*

[12] Free Software Foundation, *Variable Attributes – Using the GNU Compiler Collection (GCC)*, 2008. [En línea]. Disponible en: https://gcc.gnu.org/onlinedocs/gcc-4.3.2//gcc/Variable-Attributes.html

[13] Espressif Systems, *ESP-DSP: Ejemplo de filtro FIR.* [En línea]. Disponible en: https://github.com/espressif/esp-dsp/blob/3c12d059f96ec95ed00769fb32d41ca23e20cdf1/examples/fir/main/dsps_fir_main.c

[14] Espressif Systems, *Issue oficial sobre velocidad de la librería ESP-DSP*, GitHub. [En línea]. Disponible en: https://github.com/espressif/esp-dsp/issues/97

[15] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*, Cap. 16: Windowed-Sinc Filters. California Technical Publishing, 1997. [En línea]. Disponible en: https://www.analog.com/media/en/technical-documentation/dsp-book/dsp_book_Ch16.pdf